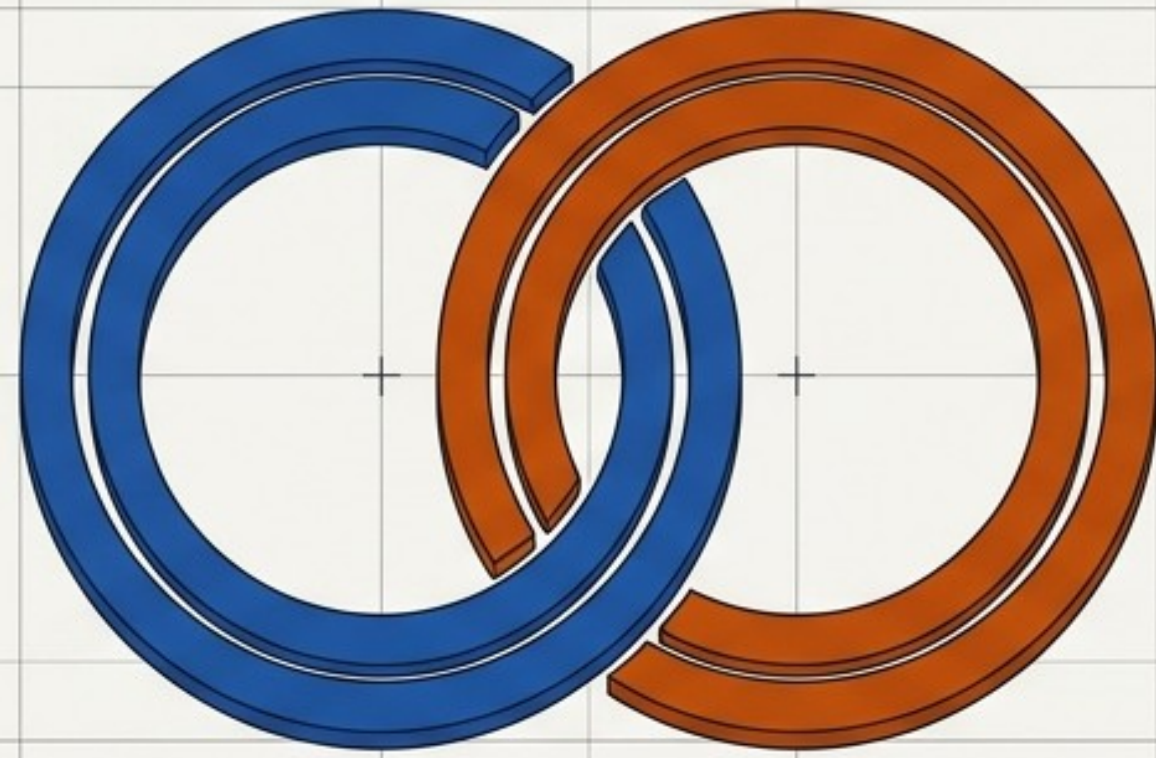




O Motor Duplo: Desenvolvimento Guiado por IA

[Discovery]

[Delivery]

[Automação]

O Fluxo de Desenvolvimento Assistido por IA: Do Discovery ao Delivery

Este guia define um processo de ponta a ponta para construir produtos digitais, começando pela validação do problema até a automação da entrega. O fluxo utiliza IAs generativas e agentes especializados para transformar especificações técnicas em código funcional e testes validados.

Fase de Discovery (Descoberta e Design)



Definição e Refinamento Técnico

Elaboração dos documentos problem.md, PRD, especificações e arquitetura utilizando prompts de IA.

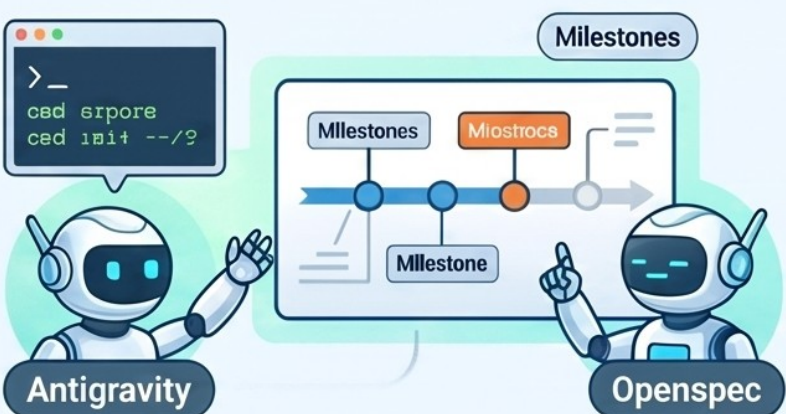
Design System e Prototipagem

Criação de interfaces no Stitch e definição de sistemas visuais via getdesign.md.

Ferramentas Essenciais utilizadas em cada fase do ciclo de vida do projeto

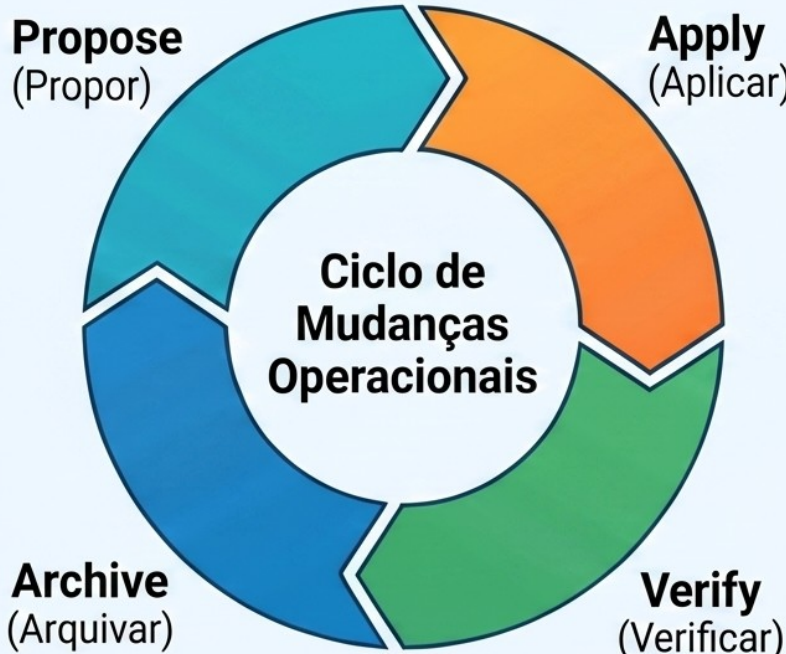
Fase	Ferramentas Chave
Discovery	ChatGPT/Claude, Stitch, GitHub
Configuração	Node.js, Docker, MCP Servers
Entrega	Antigravity, Openspec, Playwright

Fase de Delivery (Entrega e Validação)



Agentes de IA e Spec-Driven Development

Uso do Antigravity e Openspec para gerenciar o roadmap de mudanças via comandos.



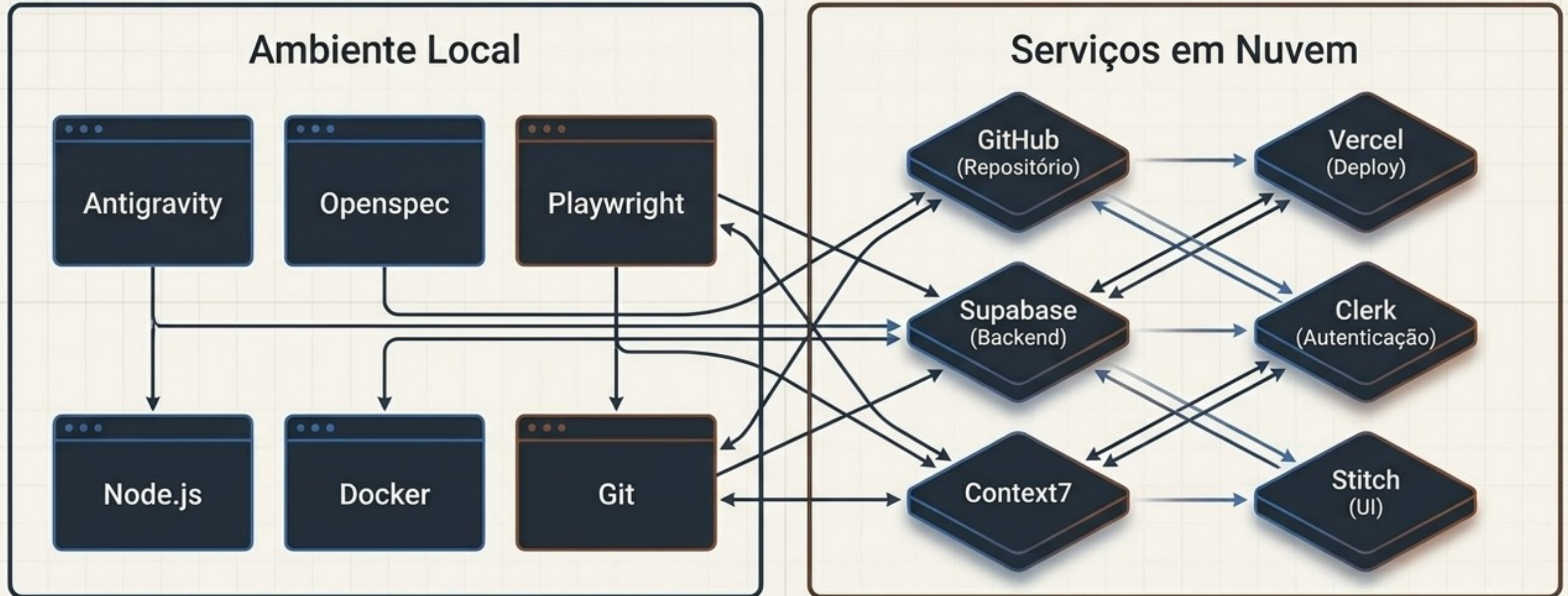
Verificação Automatizada com Playwright

Geração de planos e casos de teste automatizados baseados na especificação do projeto.

A Matriz do Motor Duplo

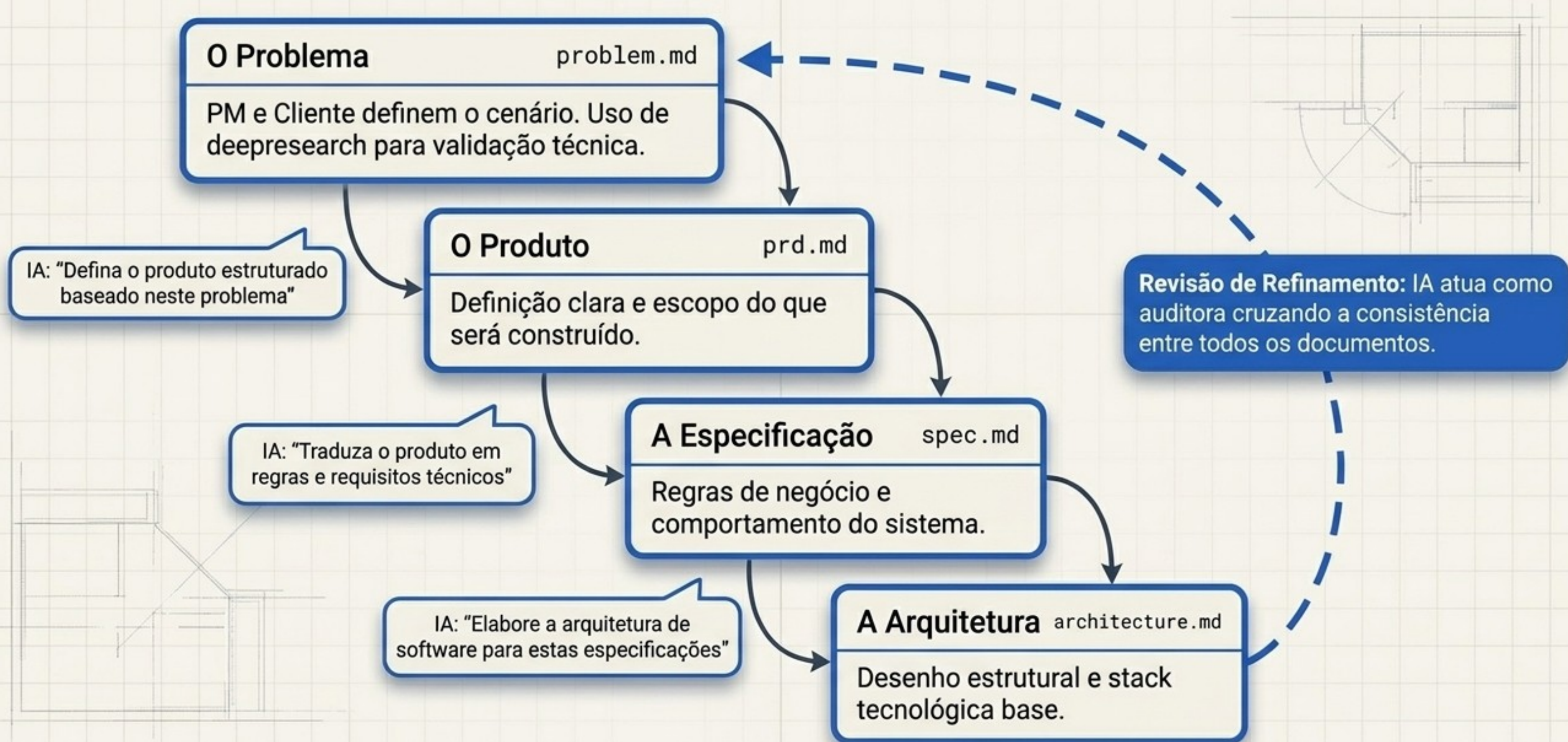
	Fase 1: Discovery	Fase 2: Delivery
Objetivo	Transformar problema em protótipo validado	Transformar roadmap em incremento testado
Participantes	PM, Cliente, UX, Dev    	UX, Dev  
Ferramentas Chave	Modelos Gerais (ChatGPT, Claude), Stitch	Agentes Específicos (Antigravity, Cursor, Openspec, Playwright)
Artefatos Gerados	prd.md, spec.md, Protótipos	Planos de Teste, Mudanças, Código, Build

O Ecossistema de Ferramentas e Serviços



Aviso Crítico: O pré-requisito central para a orquestração. Todas as credenciais de nuvem devem estar mapeadas localmente no arquivo `.env` para o funcionamento integrado dos agentes de IA.

Fase 1: A Cascata de Artefatos



Desenho de UI e Prototipação Contínua

UX Designer & Desenvolvedor

1. Design System

Base: getdesign.md
Geração via Stitch
Saída: design.md



2. Criação Base

Inputs: prd.md + spec.md
Mapeamento das regras
de negócio para
componentes visuais.



3. Exploração (Stitch)

[Preview] : Validação
rápida em nova aba
[Generate Variations] :
Exploração de alternativas
de design
[Interact] : Teste de fluxo
em protótipos gerados

Configurando o Cérebro do Agente



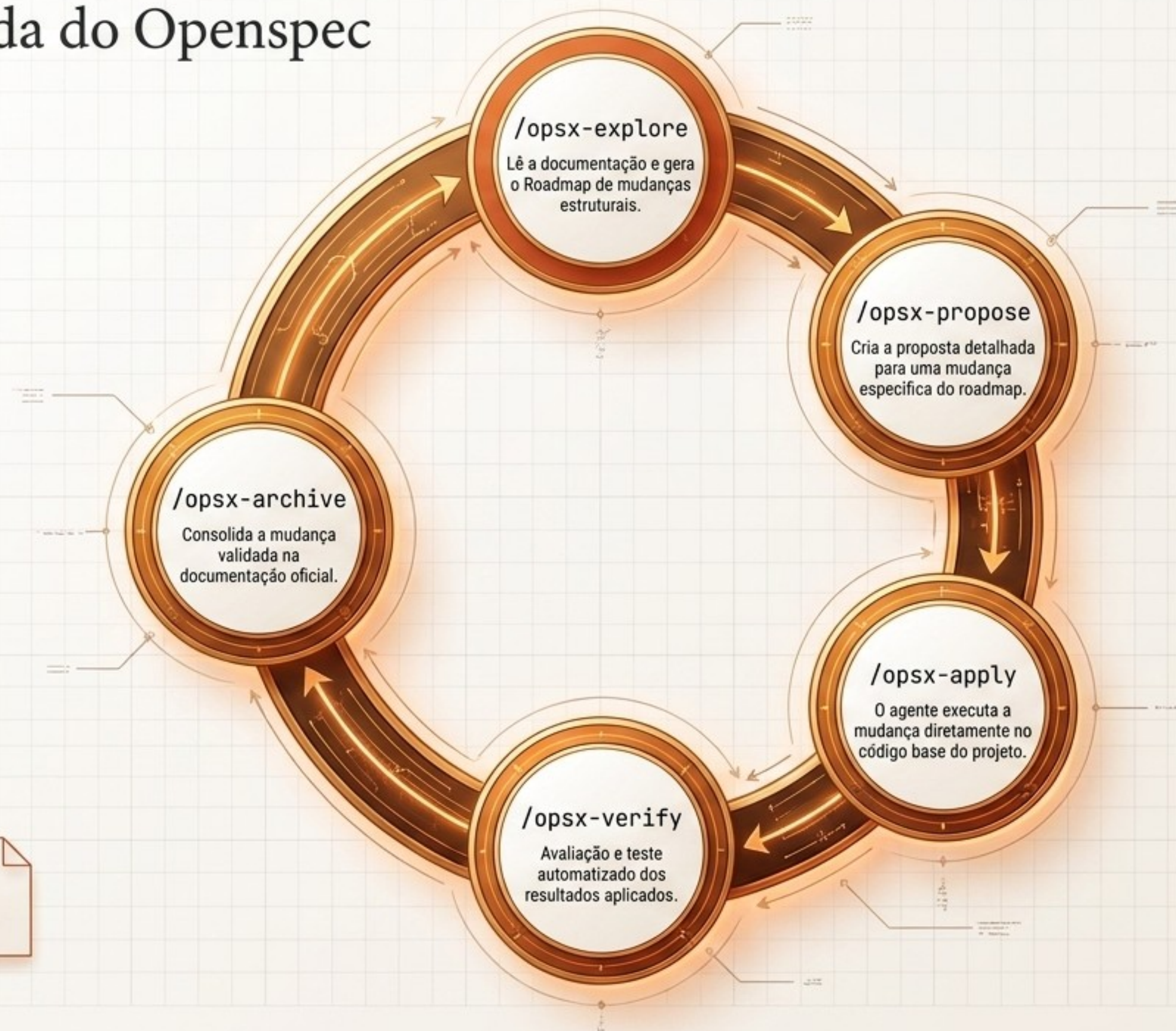
Melhor Prática: Inicie uma Nova Conversa para cada tarefa, isolando o contexto e evitando alucinações da IA.

Estratégia de Modelos: Pensar vs. Agir

	Fase de Planejamento	Fase de Execução
Objetivo	Estruturar a solução e arquitetura técnica	Implementar o código base e escrever testes
Tipo de Modelo	Alto Raciocínio / Modelos 'Thinking'	Modelos Rápidos / Focados em Tarefas
Ação Central	Criação do artefato <code>implementation_plan.md</code>	Execução sequencial passo a passo do plano gerado
Ajuste de Prompt	Uso de <code><></code> para valores específicos e <code>@</code> para referenciar arquivos	Interação reativa e rápida para resolução de erros de linting

Transição orientada por
`implementation_plan.md`

O Ciclo de Vida do Openspec



Ferramentas SDD Compatíveis:
Openspec, BMAD, Spec-kit, GSD.

Estruturando Mudanças e Propostas

A Proposta: /opsx-propose

```
1  openspec/
2
3  |
4  |  changes/
5  |
6  |  |  01-estrutura-inicial/
7  |  |
8  |  |  |  proposal.md
9  |  |  |
10 |  |  |  tasks.md
11 |  |  |
12 |  |  |  context.md
13 |  |
14 |  |
15 |  |
16 |  |
17 |  |
18 |  |
19 |  |
20 |  |
21 |  |
22 |  |
23 |  |
24 |  |
```

A Execução: /opsx-apply

> Aplique a mudança descrita em 01-estrutura-inicial

processing tasks.md...

```
...
generating core files...
{
  cuments: [""],
  ..
  data: [
    {
      propost,
      return..
    },
  ],
}
}
```

```
src/core/auth.js {
  const po = {
    input: "autocomzctant.js",
    name: "tast",
  };
}
```

```
src/components/App.js {
  const poun=ectapp = {}
  return name "../App">?^";
},
}
```

```
src/styles/global.css {
  styles == "global.css";
};
```

Anotação Estratégica: A estrutura inicial e a autenticação (ex: Clerk) devem ser as primeiras entregas do roadmap para pavimentar a base sólida do sistema.

O Pipeline de Verificação

Agentes Inteligentes

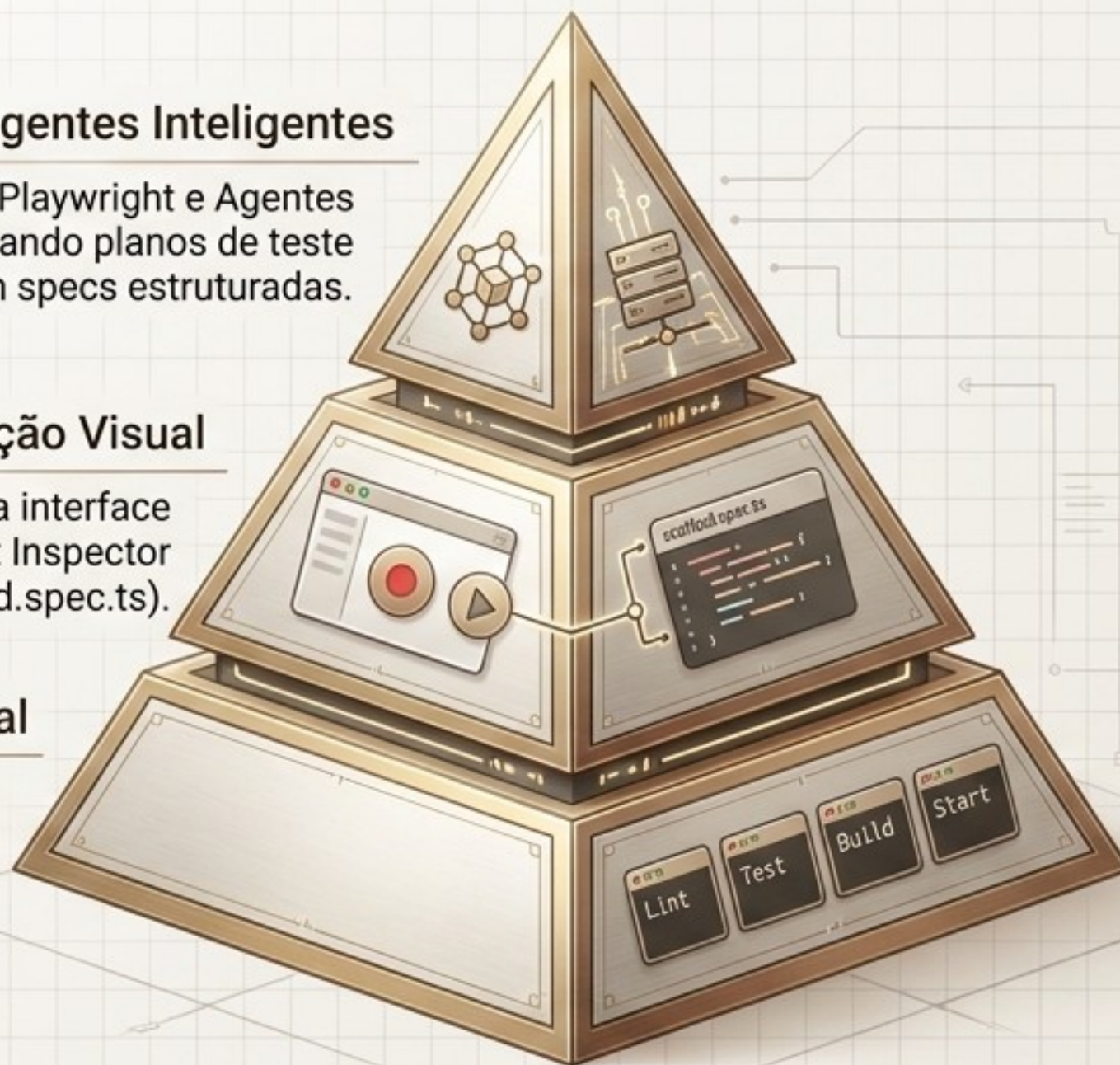
MCP Server do Playwright e Agentes autônomos criando planos de teste baseados em specs estruturadas.

Automação Visual

Gravação de interações da interface via Playwright Inspector (scaffold.spec.ts).

Verificação Manual

Comandos diretos e fundamentais de terminal: Lint, Test, Build, Start.



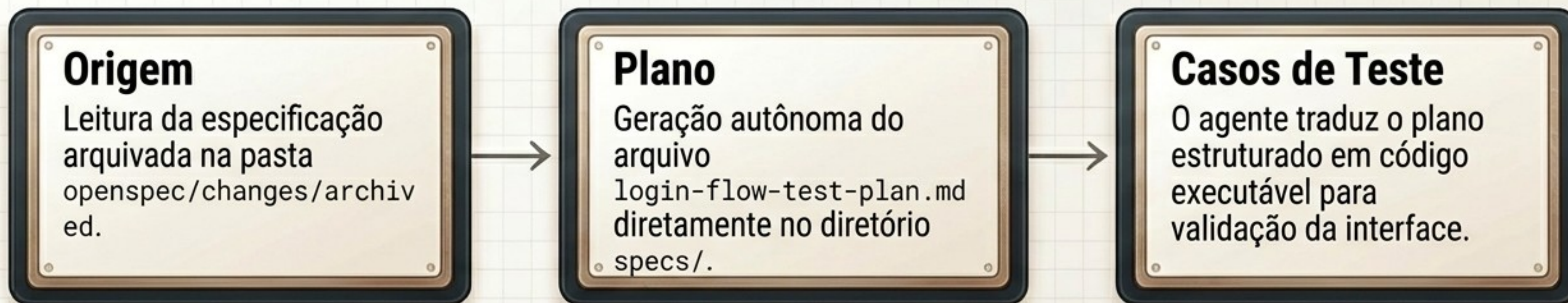
A cada mudança aplicada, o pipeline de verificação atua como um portão de qualidade indispensável antes do arquivamento.

Automação de UI com Playwright



Integração Antigravity: Copiar os arquivos gerados da pasta `.opencode` para `.agents` para garantir compatibilidade total.

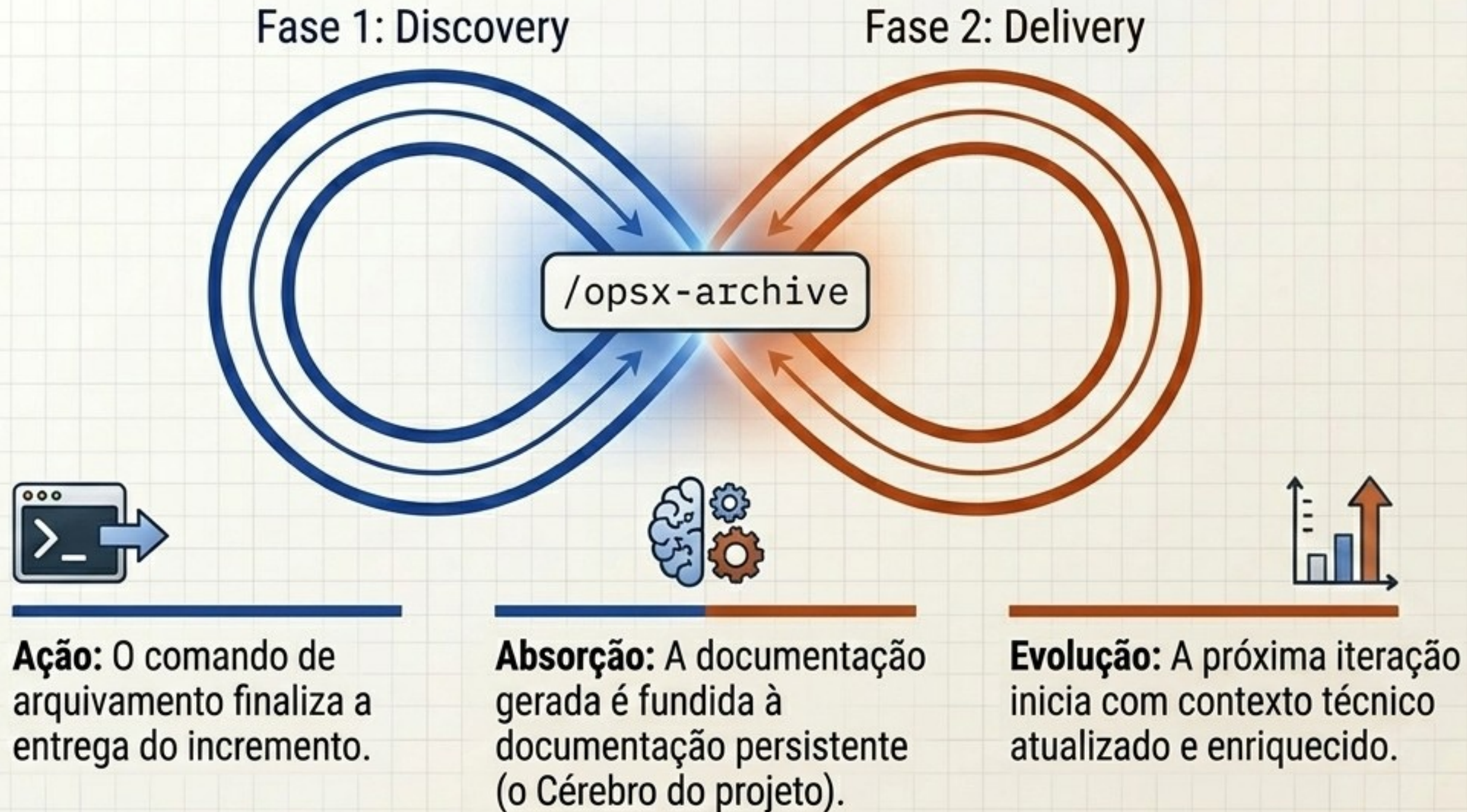
Testes Inteligentes e Baseados em Contexto



O Fator Clerk: Restrições Reais

Para testar fluxos de login complexos, o agente deve ser instruído a usar credenciais validadas previamente (usuário/senha reais no prompt) e executar testes com acompanhamento visual contínuo.

A Síntese: O Loop Contínuo de Entrega



O arquivamento fecha o ciclo. Cada incremento validado atualiza a base de conhecimento, tornando o Agente exponencialmente mais preciso.